DICE ROLLING SIMULATOR

Python Project Report

Project Type: Beginner Python Programming Project
Programming Language: Python 3
Framework: Streamlit
Main File: dice.py

This report documents a simple web-based dice rolling simulator built with Python and Streamlit. The app shows a title, a "Roll Dice" button, and an ASCII-art die face. It is designed to represent a standard six-sided die. The report also records what the submitted source file actually does when it is run, and includes a statistical experiment on the random-number method the simulator is meant to use.

1. Introduction

A dice rolling simulator is a small programming exercise that models a familiar real-world process. A standard six-sided die has six possible outcomes: 1, 2, 3, 4, 5 and 6. For a fair die, each outcome has an equal theoretical probability of 1/6, or about 16.67%.

This project presents the die in a browser using Streamlit, a Python library for building web apps. The user clicks a button and the program displays a die face drawn with text characters.

2. Objectives

The main objectives of the project are to:

- Build a simple web interface in Python using Streamlit.
- Represent the six faces of a die as ASCII art stored in a dictionary.
- Display a die face when the user clicks a button.
- Practice functions, dictionaries, string handling and event-driven (button) interaction.
- Examine whether the intended random-number method, randint(1, 6), gives an even distribution over many rolls.

3. Tools and Technologies

Tool	Purpose
Python 3	Language used to implement the simulator.
Streamlit	Web framework providing st.title, st.header, st.button and st.text.
random module	Imported with 'from random import *'. Intended to supply randint(1, 6).
ASCII art	Text characters used to draw the six die faces.
Dictionary	Maps each face number (1 to 6) to its ASCII-art string.

4. Program Methodology

The program is written as a Streamlit script, which runs from top to bottom each time the user interacts with the page:

1. Import the random module and Streamlit.
2. Display the title 'Dice Simulator 1.0' and the header 'Click the button to roll the dice.'
3. Define dice_roll(), which holds a dictionary of six ASCII-art faces and returns one of them.
4. Show a 'Roll Dice' button.
5. When the button is clicked, call dice_roll() and show the returned text with st.text().

5. Important Code

```
from random import *
import streamlit as st

st.title('Dice Simulator 1.0')
st.header('Click the button to roll the dice.')

def dice_roll():
    num=1
    dice={
        1:(" [---------]\n "
           "    |         |\n "
           "    |    o    |\n "
           "    |         |\n "
           "    [---------]\n "),
        2:( ... ),   # faces 2 to 6 follow the same pattern
        ...
    }
    return dice[num]

if st.button('Roll Dice'):
    run=dice_roll()
    st.text(run)
```

6. Code Explanation

Imports. 'from random import *' brings in the random module's functions, including randint(). Streamlit is imported as st.

Page elements. st.title() and st.header() draw the heading text. st.button('Roll Dice') returns True on the run in which the button was clicked, so the code inside the if-block runs only after a click.

dice_roll(). The function builds a dictionary whose keys are the numbers 1 to 6 and whose values are multi-line strings drawing each face. It then returns dice[num]. The variable num selects which face is returned.

Displaying the result. st.text() shows the returned string in a fixed-width font, which is what lets the ASCII art line up on screen.

7. Observed Program Behavior

To check what the submitted code does, the dice_roll() function was executed directly (with Streamlit's display calls replaced by empty stubs) and its output was inspected. The findings are listed below.

Finding 1: the result does not change. In the submitted file, num is set to the fixed value 1 and is never reassigned. The imported randint() function is never called. As a result, dice_roll() returns the face for 1 every time. Calling it 10,000 times produced exactly one distinct output, the face for 1.

Face	Times returned (10,000 calls)	Percentage	Expected % (fair die)
1	10,000	100.00%	16.67%
2	0	0.00%	16.67%
3	0	0.00%	16.67%
4	0	0.00%	16.67%
5	0	0.00%	16.67%
6	0	0.00%	16.67%

Finding 2: output of face 1 as currently produced. The returned text is:

```
 [---------]
     |         |
     |    o    |
     |         |
     [---------]
<end of string>
```

The top bracket line is indented by one space, while the other lines are indented by five, so the top of the die is slightly misaligned.

Finding 3: faces 2 to 6 are stored without line breaks. Only face 1 contains newline characters. Faces 2 to 6 are written as one continuous line of text, so st.text() would show each of them as a single long line rather than a five-line die. For example, face 2 is stored as:

```
" [---------]     |         |     |   o o   |     |         |     [---------] "
```

Finding 4: some pip layouts are not standard. Faces 2 and 3 place their dots side by side in the middle row, and face 4 uses a 1-2-1 arrangement. Faces 5 and 6 match the usual layouts.

8. What Would Make the Simulation Fair?

The intended design of a dice simulator is to choose an integer from 1 through 6 using Python's pseudo-random number generator, for example `num = randint(1, 6)`. Under uniform selection, the six outcomes have the same theoretical probability:

Expected probability of each face = 1/6, about 16.67%

Fairness should not be judged from only a few rolls. For example, obtaining several 6s in a row is possible even when the underlying process is fair. A larger sample gives a better view of the overall distribution.

9. Statistical Experiment: 10,000 Rolls of randint(1, 6)

A separate experiment was run to check the random-number method the simulator is meant to use. Python's `random.randint(1, 6)` was called 10,000 times with the random seed set to 42, so the results can be reproduced. **Note:** this tests randint(1, 6) on its own. It does not test the submitted dice.py, which currently does not call randint (see Section 7).

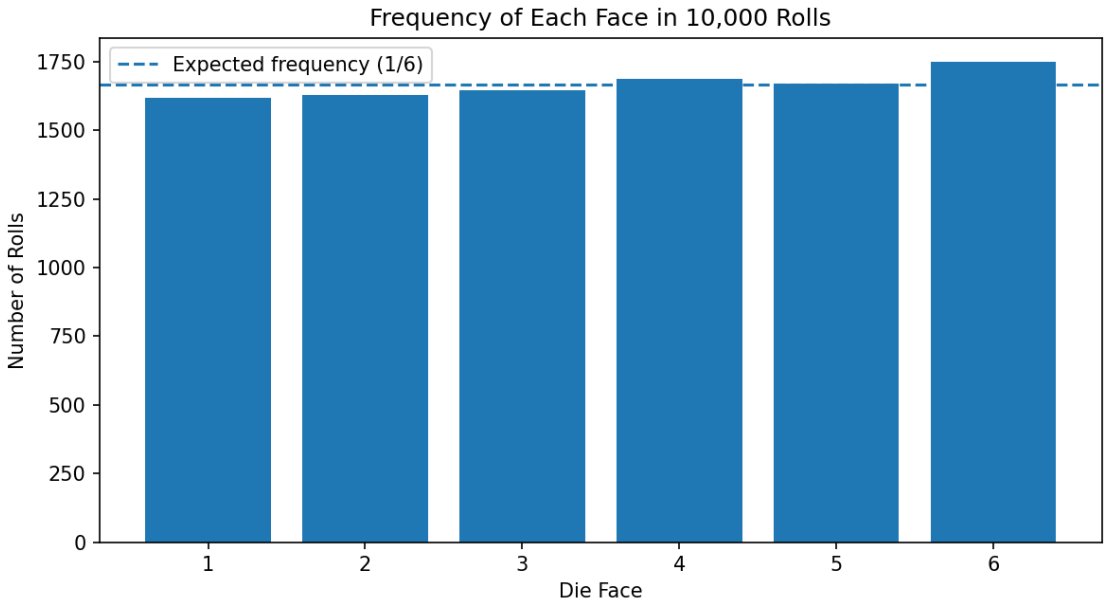
Face	Observed Rolls	Observed %	Expected %	Difference
1	1,618	16.18%	16.67%	-0.49
2	1,629	16.29%	16.67%	-0.38
3	1,647	16.47%	16.67%	-0.20
4	1,687	16.87%	16.67%	+0.20
5	1,671	16.71%	16.67%	+0.04
6	1,748	17.48%	16.67%	+0.81

Chi-square goodness-of-fit test. With an expected count of 1,666.67 per face, the chi-square statistic is 6.73 with 5 degrees of freedom, giving a p-value of about 0.24. The usual 5% critical value for 5 degrees of freedom is 11.07, so the result is well inside the range expected from random variation. The test finds no evidence that the faces are unevenly distributed.

Effect of sample size. Using the first n rolls of the same 10,000-roll sequence, the largest gap between any face's percentage and 16.67% shrinks as n grows:

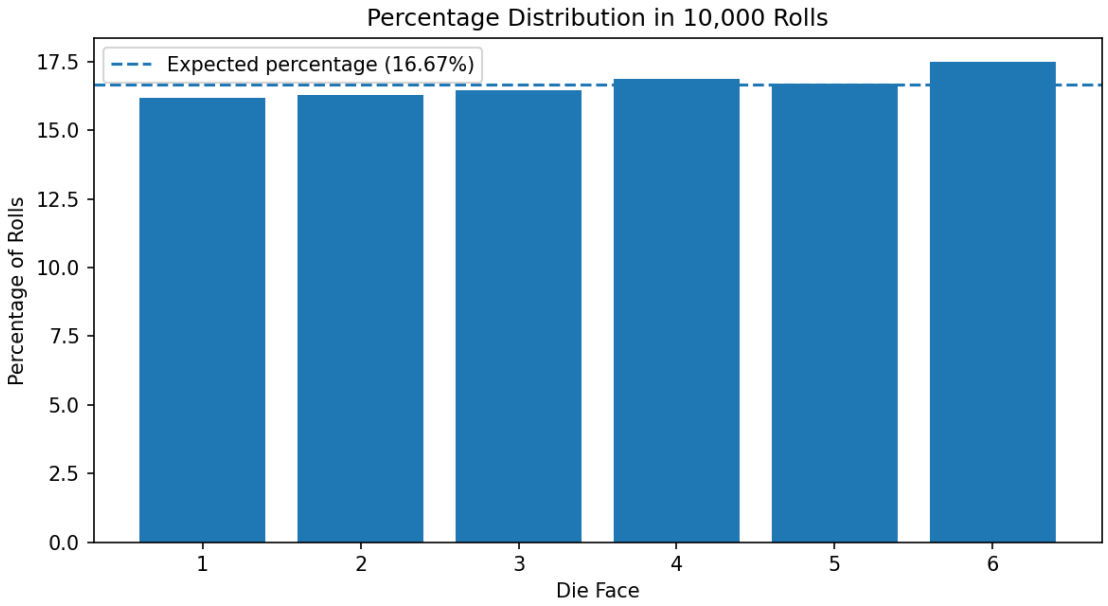
Number of rolls (n)	Largest gap from 16.67%
60	8.33 percentage points
600	2.67 percentage points
6,000	1.12 percentage points
10,000	0.81 percentage points

10. Frequency Distribution



The dashed reference line shows the theoretical expected frequency of 1,666.67 occurrences per face. The observed bars differ slightly, as expected from random variation.

11. Percentage Distribution



The percentage chart compares the observed share of each face with the theoretical 16.67% share. Small deviations are expected in a random experiment.

12. Interpretation of Results

The experiment shows that Python's randint(1, 6) behaves like a fair six-sided die over a large sample. A fair random process does not need to produce exactly the same number of each face. What matters is that the differences are small and not systematic.

The experiment is evidence about the random-number method, not about the submitted app. The app, as written, returns face 1 on every click (Section 7), so its output is not random. The experiment also does not prove that every individual roll is fair, and it does not apply to a physical die.

13. Limitations

- The displayed face never changes, because num is fixed at 1 and randint() is not called.
- Faces 2 to 6 are stored as single lines and would not display as a boxed die.
- Face 1 has a misaligned top edge, and the pip layouts for 2, 3 and 4 are non-standard.
- 'from random import *' imports everything from the module, while only randint is needed.
- The app shows one die only and keeps no history or statistics of past rolls.
- No input validation or automated tests are included (the interface has only one button, so validation is minimal).
- It relies on pseudo-randomness, not a physical die.

14. Future Improvements

Suggested fixes and extensions:

- Replace `num=1` with `num = randint(1, 6)` and import it explicitly with `from random import randint`.

- Rewrite faces 2 to 6 with the same line breaks and indentation as a corrected face 1, using standard pip layouts.
- Build each face from a list of lines joined with newlines to keep the alignment consistent.
- Add a roll counter and a frequency table using st.session_state.
- Show a bar chart of results directly in the app with Streamlit's charting functions.
- Let the user roll several dice at once or choose dice with different numbers of sides.
- Add automated unit tests, for example checking that dice_roll() can return all six faces.

15. Conclusion

The Dice Rolling Simulator is a compact Streamlit project that demonstrates basic Python concepts: functions, dictionaries, string formatting and a button-driven interface. In its current form the interface works, but the face shown is always 1 because the random number is never generated, and faces 2 to 6 are not formatted for display. The 10,000-roll experiment shows that randint(1, 6), once connected, would give frequencies close to the expected 16.67% for each face, with normal random variation. Applying the fixes in Section 14 would make the program match its intended design.

End of Project Report